

Constraint Satisfaction Problem using Branch and Bound and Backtracking for Graph Coloring

Aim

To implement a solution for a Constraint Satisfaction Problem (CSP) using Branch and Bound and Backtracking for the Graph Coloring Problem.

Theory

Constraint Satisfaction Problem (CSP)

A Constraint Satisfaction Problem (CSP) is a problem in which:

- A set of variables is given
- Each variable has a domain of possible values
- Constraints restrict the combinations of values the variables can take

The objective is to assign values to all variables while satisfying all constraints.

Components of CSP

1. Variables
 2. Domains
 3. Constraints
-

Graph Coloring Problem

Graph coloring is a CSP in which colors are assigned to graph vertices such that no two adjacent vertices have the same color.

The goal is to color the graph using a minimum number of colors.

Applications of Graph Coloring

- Timetable scheduling
 - Register allocation in compilers
 - Frequency assignment
 - Map coloring
 - Resource allocation
-

Backtracking

Backtracking is a recursive problem-solving technique that:

- Tries one possible solution
- Checks whether it satisfies constraints
- If not, returns back and tries another possibility

It follows:

Try → Check → Backtrack if needed

Steps of Backtracking

1. Select a vertex
 2. Assign a color
 3. Check whether assignment is safe
 4. If safe, move to next vertex
 5. If not safe, try another color
 6. If no color works, backtrack
-

Branch and Bound

Branch and Bound is an optimization technique used to reduce unnecessary searching.

In Graph Coloring

- Branching means trying different color assignments
- Bounding means stopping exploration when constraints are violated

This improves efficiency by pruning invalid branches early.

Algorithm

Algorithm for Graph Coloring using Backtracking

1. Start with first vertex
 2. Assign a color from 1 to m
 3. Check if color assignment is safe
 4. If safe, recursively color next vertex
 5. If all vertices are colored, print solution
 6. If no color is possible, backtrack
-

Flowchart

Start

↓

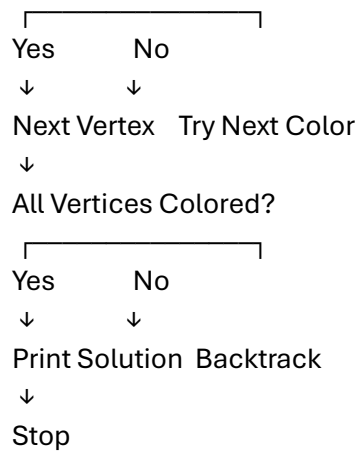
Input Graph and Colors

↓

Assign Color to Vertex

↓

Is Color Safe?



Program

Python Program

```
def is_safe(graph, color, v, c):
    for i in range(len(graph)):
        if graph[v][i] == 1 and color[i] == c:
            return False
    return True

def solve_graph_coloring(graph, m, color, v):
    if v == len(graph):
        return True

    for c in range(1, m + 1):
```

```

if is_safe(graph, color, v, c):

    color[v] = c

    if solve_graph_coloring(graph, m, color, v + 1):
        return True

    # Backtracking
    color[v] = 0

return False

def graph_coloring(graph, m):

    color = [0] * len(graph)

    if solve_graph_coloring(graph, m, color, 0):

        color_names = ["", "Red", "Blue", "Green", "Yellow"]

        print("Solution Exists:\n")

        for i in range(len(color)):
            print("Vertex", i, "----> ", color_names[color[i]])

    else:
        print("No Solution Exists")

# Adjacency Matrix
graph = [
    [0,1,1,1],
    [1,0,1,0],
    [1,1,0,1],
    [1,0,1,0]
]

m = 3

graph_coloring(graph, m)

```

Input

Graph:

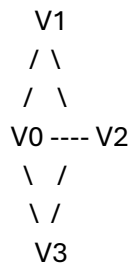
```

0 1 1 1
1 0 1 0
1 1 0 1
1 0 1 0

```

Number of Colors = 3

Graph Diagram



Output

Solution Exists:

Vertex 0 ---> Red

Vertex 1 ---> Blue

Vertex 2 ---> Green

Vertex 3 ---> Blue

Working of Backtracking

Step 1

Assign Red to V0

Step 2

Assign Blue to V1 because Red causes conflict

Step 3

Assign Green to V2 because Red and Blue conflict

Step 4

Assign Blue to V3

If any vertex cannot get a valid color:

- Remove previous assignment
- Return to previous vertex
- Try another color

This process is called Backtracking.

Time Complexity

Worst Case:

$$O(m^V)$$

Where:

- m = number of colors
- V = number of vertices

Advantages

- Simple and effective
- Finds valid solution systematically
- Avoids invalid solutions early
- Useful for CSP problems

Disadvantages

- Time complexity is high for large graphs
 - Recursive calls increase computation
-

Result

Thus, the Graph Coloring Problem was successfully implemented using Constraint Satisfaction Problem techniques with Backtracking and Branch and Bound.

Different input

Enter number of vertices: 5

Enter adjacency matrix:

```
graph = [  
    [0, 1, 1, 0, 0],  
    [1, 0, 1, 1, 0],  
    [1, 1, 0, 1, 1],  
    [0, 1, 1, 0, 1],  
    [0, 0, 1, 1, 0]
```

Enter number of colors: 3